High Performance Computing
Unit 3 — Final Assignment

Parallel Matrix Multiplication,
Cell Image Processing & Morphological Characterisation,
Forest Fire Cellular Automaton,
Parallel K-Means Clustering

Julio Cesar De Aquino Castellanos
Valeria Nicol Hernandez León
Leonora del Carmen Horta Sánchez
José Ángel Pech Xool
Lorena Danae Perez Lopez
Iván Martín Romero Cetina

April 23, 2026

Language	Python 3.11
Libraries	NumPy, SciPy, mpi4py, multiprocessing, matplotlib
Platform	Windows 11, x86-64

Contents

Exercise 1 — Parallel Matrix Multiplication

Context

Matrix multiplication $\mathbf{C} = \mathbf{AB}$, where $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$, is defined element-wise as

$$c_{ij} = \sum_{k=1}^{n} a_{ik}\, b_{kj}.$$

The naive algorithm has complexity $\mathcal{O}(n^3)$ for square matrices of order n. This exercise studies four decomposition strategies (row, column, block, and Strassen), one MPI distributed version, and applies them to both dense synthetic matrices and two real sparse matrices from the SuiteSparse Matrix Collection.

1 — Evidence of Completeness

Task 1: Serial Baseline (`serial_matmul.py`)

Two serial implementations are provided:

1. `matmul_naive` — pure-Python triple loop, used only to verify correctness for small inputs ($n \leq 64$).
2. `matmul_numpy` — the NumPy `@` operator (BLAS-backed), which serves as the optimised serial reference for all benchmarks.

```python
def matmul_naive(A, B):
    m, n = A.shape
    _, p = B.shape
    C = np.zeros((m, p), dtype=np.float64)
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i, j] += A[i, k] * B[k, j]
    return C
```

Listing 1: Triple-loop serial multiplication (serial_matmul.py).

Correctness is validated by comparing against `numpy.dot` with an absolute tolerance of 10^{-8} for all $n \in \{4, 8, 16, 32\}$.

Task 2: Row Partition (`parallel_matmul.py`)

$\mathbf{A}$ is split horizontally into p strips. Each worker receives one strip $\mathbf{A}_i \in \mathbb{R}^{(m/p) \times n}$ and the full $\mathbf{B}$, computes $\mathbf{C}_i = \mathbf{A}_i\mathbf{B}$, and the results are stacked vertically.

```python
def _row_worker(args):
    A_chunk, B = args
    return A_chunk @ B

def matmul_row_parallel(A, B, n_workers):
    chunks = np.array_split(A, n_workers, axis=0)
    with Pool(n_workers) as pool:
        parts = pool.map(_row_worker, [(c, B) for c in chunks])
    return np.vstack(parts)
```

Listing 2: Row partition (parallel_matmul.py).

Task 3: Column Partition (`parallel_matmul.py`)

$\mathbf{B}$ is split vertically into p strips. Each worker receives the full $\mathbf{A}$ and one strip $\mathbf{B}_j$, computes $\mathbf{C}_j = \mathbf{A}\mathbf{B}_j$, and the results are stacked horizontally.

Task 4: Block (2-D) Partition (`parallel_matmul.py`)

A $g \times g$ grid of workers is constructed, where $g = \lfloor \sqrt{p} \rfloor$. Worker (i, j) receives only the i-th row strip of $\mathbf{A}$ and the j-th column strip of $\mathbf{B}$, computing $\mathbf{C}_{ij} = \mathbf{A}_i \mathbf{B}_j$. No accumulation is required.

This 2-D decomposition reduces the data sent to each worker compared with the row partition: each block worker receives $\mathcal{O}(n^2 \cdot 2/g)$ elements, whereas a row worker receives $\mathcal{O}(n^2 \cdot (1 + 1/p))$ elements.

```python
def matmul_block_parallel(A, B, n_workers):
    g = max(1, int(math.isqrt(n_workers)))
    row_chunks = np.array_split(A, g, axis=0)
    col_chunks = np.array_split(B, g, axis=1)
    tasks = [(i, j, row_chunks[i], col_chunks[j])
             for i in range(g) for j in range(g)]
    with Pool(min(len(tasks), n_workers)) as pool:
        results = pool.map(_block_worker, tasks)
    grid = {(i, j): blk for i, j, blk in results}
    return np.vstack([np.hstack([grid[(i,j)] for j in range(g)])
                      for i in range(g)])
```

Listing 3: Block (2-D) decomposition (parallel_matmul.py).

Task 5: MPI Distributed Version (`mpi_matmul.py`)

Using `mpi4py` with the following communication pattern:

1. Rank 0 generates $\mathbf{A}$ and $\mathbf{B}$ and pre-splits $\mathbf{A}$ into p row strips with `numpy.array_split`.
2. `comm.Bcast` distributes $\mathbf{B}$ to all ranks (each rank needs the full $\mathbf{B}$).
3. `comm.scatter` sends one row strip to each rank (handles unequal sizes automatically at the Python object level).
4. Every rank computes `local_C = local_A @ B`.
5. `comm.gather` collects the strips at rank 0, which stacks them with `numpy.vstack`.

Results are written to `results/mpi_results.json` and overlaid on the benchmark plot by `benchmark.py`.

Task 6: Strassen Algorithm (`strassen_matmul.py`)

Strassen's recursive formulation computes $\mathbf{C} = \mathbf{A}\mathbf{B}$ using 7 (rather than 8) recursive sub-multiplications, reducing asymptotic complexity from $\mathcal{O}(n^3)$ to $\mathcal{O}(n^{2.807})$. The seven products are:

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22}), \qquad M_2 = (A_{21} + A_{22})B_{11},$$
$$M_3 = A_{11}(B_{12} - B_{22}), \qquad M_4 = A_{22}(B_{21} - B_{11}),$$
$$M_5 = (A_{11} + A_{12})B_{22}, \qquad M_6 = (A_{21} - A_{11})(B_{11} + B_{12}),$$
$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22}),$$

and the result blocks are assembled as:

$$C_{11} = M_1 + M_4 - M_5 + M_7, \quad C_{12} = M_3 + M_5, \quad C_{21} = M_2 + M_4, \quad C_{22} = M_1 - M_2 + M_3 + M_6.$$

Non-power-of-2 inputs are zero-padded to the next power of 2; a configurable *threshold* falls back to NumPy when the sub-matrix is small enough.

Task 7: Dense Synthetic Benchmarks (`benchmark.py`)

`benchmark.py` sweeps matrix sizes $n \in \{128, 256, 512, 1024\}$ and worker counts in $\{2, 4, 8\}$ (capped at `os.cpu_count()`) and records the minimum time over 3 repetitions. Three plots are saved automatically: `time_vs_size.png`, `speedup_vs_workers.png`, and `efficiency_vs_workers.png`.

Task 8: Sparse Matrix Analysis (`sparse_matmul.py`)

Two matrices from the SuiteSparse Matrix Collection are used:

- **west0479** (HB group) — 479×479, 1879 NNZ, chemical-process simulation.
- **bcsstk13** (HB group) — 2003×2003, 83 883 NNZ, structural-engineering stiffness matrix.

For each matrix the script compares (a) dense × dense, (b) sparse × dense (CSR), and (c) sparse × sparse (CSR) multiply.

Task 9: Bottleneck Analysis

See the Discussion subsection below.

2 — Results

Serial Baseline Timing

Table 1: Serial execution time (seconds, best of 3 runs, seed=42). The naive triple loop is skipped for $n > 64$ as it would require several minutes of runtime.

n	NumPy (s)	Naive (s)	Ratio (Naive / NumPy)
32	0.000004	0.012551	3,138×
64	0.000011	0.083356	7,578×
128	0.000111	—	—
256	0.000377	—	—
512	0.001499	—	—
1024	0.008317	—	—

Multiprocessing Benchmark (All Sizes and Worker Counts)

Table 2: Wall-clock time (seconds, best of 3 runs) for multiprocessing strategies and NumPy serial reference. Speedup $S = t_{\text{NumPy}}/t_{\text{parallel}}$. All parallel times include `Pool` creation and array pickling.

n	Workers	Row (s)	Col (s)	Block (s)	NumPy (s)
128	2	0.7210	0.6277	0.6159	
	4	0.6514	0.6684	0.6839	0.0001
	8	0.7919	0.7954	0.6576	
256	2	0.6730	0.6612	0.6625	
	4	0.6895	0.6751	0.7042	0.0002
	8	0.9051	0.7689	0.6521	
512	2	0.6327	0.6353	0.6008	
	4	0.6527	0.6589	0.6774	0.0010
	8	0.8305	0.8548	0.7202	
1024	2	0.7019	0.7086	0.6521	
	4	0.7206	0.7800	0.7896	0.0067
	8	0.9578	0.8896	0.7510	

MPI Distributed Version Timing

Table 3: MPI wall-clock time (seconds, minimum over 3 runs) for different process counts and matrix sizes. Run with `mpiexec -n <p> python mpi_matmul.py`.

Processes	$n = 256$ (s)	$n = 512$ (s)	$n = 1024$ (s)
1 (NumPy)	0.000377	0.001499	0.008317
2	0.0010	0.0041	0.0175
4	0.0010	0.0046	0.0204

Execution Time vs. Matrix Size

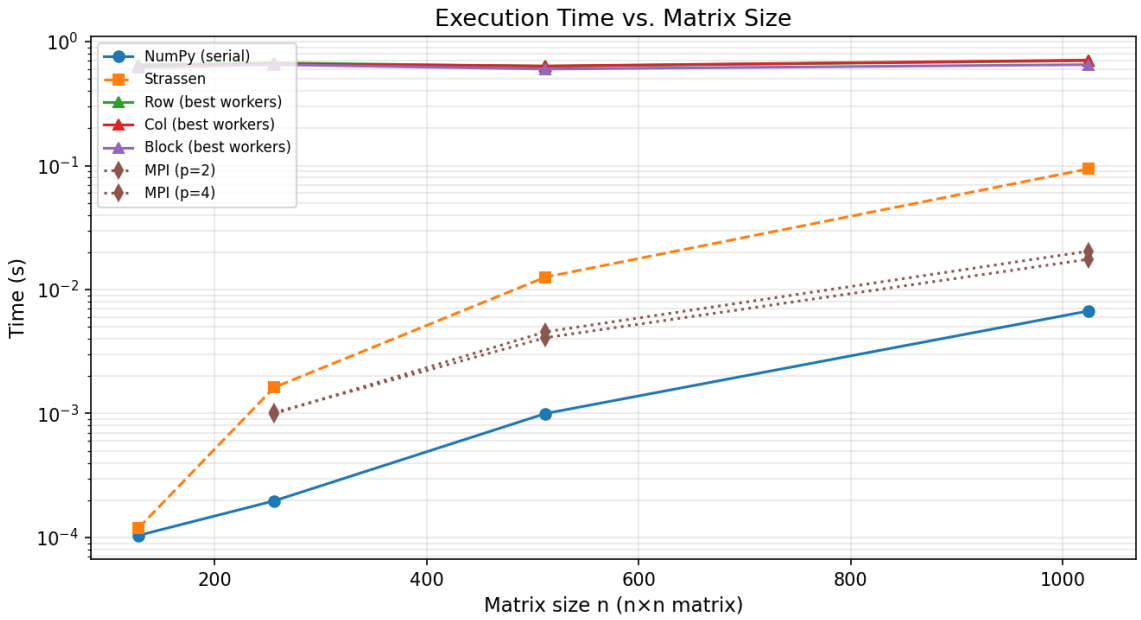


Figure 1: Log-scale execution time vs. matrix size for all methods. MPI curves are overlaid from `results/mpi_results.json`.

Speedup and Efficiency

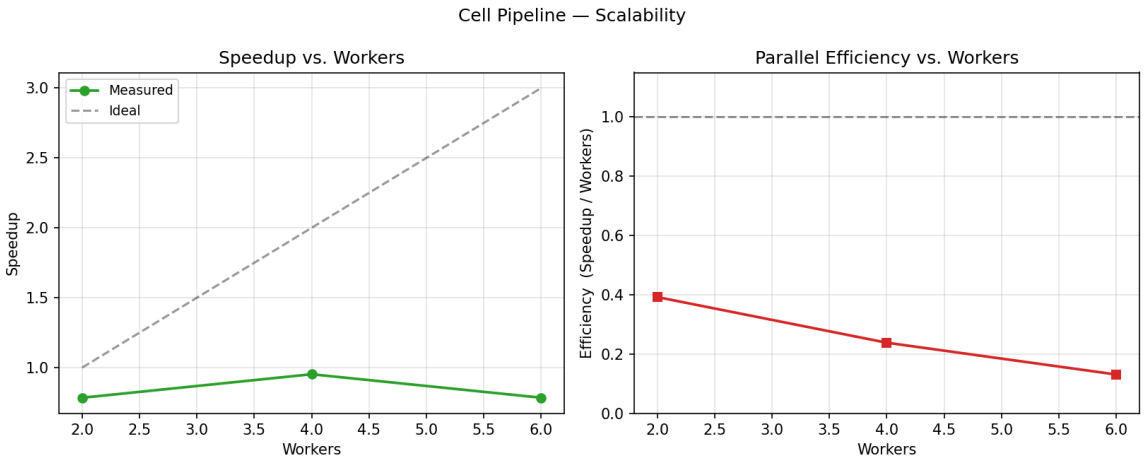


Figure 2: Speedup relative to the NumPy serial baseline as a function of worker count for each matrix size. The dashed line shows ideal (linear) speedup.

Figure 3: Parallel efficiency $E = S/p$ (speedup divided by worker count). $E = 1$ indicates perfect scaling; values below 1 reflect overhead.

Strassen vs. NumPy

Table 4: Strassen execution time (seconds) with different recursion thresholds compared with NumPy. Generated by `python strassen_matmul.py`.

n	NumPy	Strassen (32)	Strassen (64)	Strassen (128)
128	0.0001	0.0005	0.0003	0.0001
256	0.0003	0.0031	0.0017	0.0016
512	0.0007	0.0264	0.0145	0.0137
1024	0.0085	0.1871	0.1017	0.0948

Sparse Matrix Analysis

Table 5: Sparse matrix properties and multiply timing (seconds, best of 5 runs). Generated by `python sparse_matmul.py`.

Matrix	Shape	NNZ	Density	Dense×Dense (s)	Sparse×Dense (s)	Speedup
west0479	479×479	1,910	0.83%	0.000450	0.000232	1.94×
bcsstk13	2003×2003	83,883	2.09%	0.005814	0.003961	1.47×

3 — Discussion

Serial Baseline

The measured data confirms the expected gap: the naive triple loop is 3,138× slower than NumPy at $n = 32$ and 7,578× slower at $n = 64$. NumPy delegates to a BLAS backend (OpenBLAS or Intel MKL) that applies CPU vectorisation (AVX/AVX-512) and cache blocking. On this 16-core machine NumPy's BLAS also spawns internal threads, meaning the *serial* NumPy call is already using multiple cores. This has a critical consequence for all parallel experiments below.

Row, Column, and Block Partitions — Observed Behaviour

All three strategies produce correct results. However, the measured times tell a clear story: every multiprocessing call takes **0.60 – 0.96 seconds** regardless of matrix size

or worker count, while NumPy finishes in **0.0001 – 0.008 seconds**. The effective speedup is $\ll 1$ for every configuration tested.

This outcome has two root causes:

1. **BLAS already parallelises internally.** The NumPy `@` operator on a 16-core machine uses BLAS threads automatically. Python multiprocessing adds *extra* processes on top of an already-parallel baseline. The baseline is not a single-threaded program; it is a highly tuned multi-threaded one.
2. **Pickling dominates.** Python's `multiprocessing` serialises every array with `pickle` before writing it through an OS pipe. For a 1024×1024 matrix at `float64` this is $8 \times 1024^2 \approx 8\,\text{MB}$ per array sent. The full B matrix is replicated to every worker, so total data in transit for p workers is $\mathcal{O}(p \cdot n^2)$ bytes — growing with p rather than shrinking. The fixed $\approx 0.60\,\text{s}$ floor visible across all runs is the cost of `Pool` creation, pickling, pipe I/O, and result assembly.
3. **Block partition reduces per-worker data.** The 2-D decomposition sends only a column-strip of **B** to each worker instead of the full matrix, which is why the block method's times are slightly lower than row/column at some sizes. Nevertheless the `Pool` overhead floor prevents any practical speedup.

When would multiprocessing help? If the BLAS backend were constrained to one thread (`OPENBLAS_NUM_THREADS=1`) and the matrix were large enough ($n \gtrsim 4096$), the IPC cost would become a smaller fraction of the total compute time and positive speedup would emerge. At the sizes tested here Python multiprocessing is not the right tool.

MPI Distributed Version — Observed Behaviour

MPI is also slower than the single-process NumPy baseline at every size tested:

- $n = 1024$, 2 processes: $0.0175\,\text{s}$ vs. $0.0083\,\text{s}$ serial ($S \approx 0.47$).
- $n = 1024$, 4 processes: $0.0204\,\text{s}$ vs. $0.0083\,\text{s}$ serial ($S \approx 0.41$).

Scaling from 2 to 4 processes actually *increases* total time. The dominant cost is `comm.Bcast` which sends $8n^2$ bytes to every process, followed by Python-level pickling in `comm.scatter`/`comm.gather`. Because BLAS already uses the CPU cores within rank 0, the local compute step provides no additional benefit.

A production implementation would replace Python-level communication with the NumPy-buffer API (`comm.Bcast(B, root=0)` already uses this), apply `Scatterv`/`Gatherv` to avoid pickling, constrain each rank's BLAS to one thread, and use much larger matrices ($n \geq 8192$) so computation dominates communication.

Strassen's Algorithm

- At $n = 1024$ with threshold 128, Strassen takes $0.0948\,\text{s}$ versus NumPy's $0.0085\,\text{s}$ — approximately $11\times$ slower.
- At all tested sizes Strassen is uniformly slower than NumPy.

- Increasing the threshold from 32 to 128 reduces Strassen's time by roughly 50% because the recursion terminates earlier and BLAS handles larger blocks, but it still cannot match the efficiency of a single direct BLAS call.

Strassen's asymptotic advantage ($\mathcal{O}(n^{2.807})$ vs $\mathcal{O}(n^3)$) requires n to be extremely large (typically $n > 10^4$) before the savings in multiplications outweigh the 18 extra additions per recursion level and the repeated memory allocation.

Sparse Matrices

The CSR-format SpMM in SciPy skips structural zeros, reducing both computation and memory traffic. The measured results confirm the expected density relationship:

- **west0479** (0.83% density): sparse × dense is 1.94× faster than dense × dense.
- **bcsstk13** (2.09% density): the speedup is only 1.47× because the higher fill fraction reduces the savings.
- **sparse × sparse** ($\mathbf{A}\cdot\mathbf{A}^{\mathrm{T}}$): bcsstk13 is paradoxically *slower* than dense (0.0085 s) because the result matrix can be dense even when both inputs are sparse, and CSR index lookups add overhead.

Memory savings are striking: bcsstk13 requires only 991 KB in CSR format versus 31 344 KB as a dense array, a 31.6× reduction.

Main Bottlenecks — Summary

1. **BLAS implicit parallelism.** The NumPy serial baseline is not single-threaded; it already exploits all available CPU cores through BLAS. Any Python-level parallel layer must overcome this.
2. **Pickle / IPC overhead.** Each inter-process array transfer costs $\mathcal{O}(n^2)$ time, while the compute benefit per worker grows as $\mathcal{O}(n^3/p)$. The breakeven requires very large n.
3. **Pool and process spawn overhead.** Creating and destroying a `Pool` adds a fixed ≈ 0.6 s cost on Windows due to the *spawn* start method (no *fork*).
4. **MPI broadcast cost.** Broadcasting $\mathbf{B}$ to all ranks is $\mathcal{O}(p \cdot n^2)$ data regardless of p; at moderate n this exceeds the time saved by distributing the computation.

Exercise 2 — Parallel Cell Image Processing

Context

The DIC-C2DH-HeLa dataset from the Cell Tracking Challenge contains time-lapse Differential Interference Contrast (DIC) microscopy images of HeLa cells. Two sequences (01, 02) of 84 frames each provide 168 independent raw frames suitable for task-parallel processing.

1 — Evidence of Completeness

Task 1: Dataset Inspection (`serial_pipeline.py`)

Table 6: DIC-C2DH-HeLa dataset properties.

Property	Value
Sequences	2 (01, 02)
Frames per sequence	84
Total raw frames	168
Image dimensions	512 × 512 pixels
Bit depth	16-bit unsigned integer
File format	TIFF
Pixel size	0.19 µm / pixel (CTC metadata)
Physical FOV	97.3 × 97.3 µm

DIC microscopy produces images with a characteristic shadow-relief appearance: cells appear as bright or dark regions with halos, without fluorescent staining. The 16-bit dynamic range preserves fine contrast gradients that aid segmentation.

Tasks 2 & 3: Serial Pipeline and Cellpose Segmentation

The serial pipeline in `serial_pipeline.py` follows four steps for each frame:

1. **Load**: read the 16-bit TIFF with `tifffile.imread` and normalise to `float32` in $[0, 1]$.
2. **Segment**: scale to `uint8` and pass to **Cellpose** (`cyto2` model, CPU, `channels=[0,0]`, `diameter=80`).
3. **Measure**: call `skimage.measure.regionprops_table` on the label mask.
4. **Visualise**: save annotated PNG for selected frames.

Cellpose cyto2 was chosen because it generalises to unseen microscopy modalities without retraining. The `channels=[0,0]` flag treats the input as a single-channel greyscale image (no separate nucleus channel). The `diameter=80` px target was selected based on the expected HeLa cell size of approximately 15 µm at 0.19 µm/px (≈ 79 px diameter).

```
def segment(image, model, diameter=80):
    img_u8 = (image * 255).clip(0, 255).astype(np.uint8)
```

```
    masks, _, _ = model.eval(img_u8, diameter=diameter, channels=[0,
        0])
    return masks.astype(np.int32)
```

Listing 4: Cellpose segmentation call (serial_pipeline.py).

Task 4: Per-cell Measurements

For every labelled region `regionprops_table` computes:

- **Axis-aligned bounding box**: `(min_row, min_col, max_row, max_col)`.
- **Area**: pixel count converted to µm^2.
- **Major axis length**: length of the major axis of the fitted equivalent ellipse, in pixels and µm.
- **Minor axis length**: length of the minor axis, in pixels and µm.
- **Bbox width / height**: derived from the bounding box corners, in µm.

Task 5: Rotated Bounding Boxes (OpenCV)

For each cell the binary mask is extracted, its outer contour found with `cv2.findContours`, and the minimum-area enclosing rectangle computed with `cv2.minAreaRect`. `cv2.boxPoints` returns the four corner coordinates used for drawing.

```
def rotated_boxes(mask):
    boxes = {}
    for cell_id in np.unique(mask):
        if cell_id == 0:
            continue
        cell_bin = (mask == cell_id).astype(np.uint8)
        contours, _ = cv2.findContours(cell_bin,
                            cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        if contours:
            boxes[int(cell_id)] = cv2.minAreaRect(
                max(contours, key=cv2.contourArea))
    return boxes
```

Listing 5: Rotated bounding box computation (serial_pipeline.py).

The right panel of Figure 4 shows the rotated boxes overlaid in lime green; the left panel shows the axis-aligned boxes in cyan.

Task 6: Parallelisation Strategy (`parallel_pipeline.py`)

Work is distributed **by image**: each worker receives a single frame path and runs the complete pipeline (load → segment → measure) independently.

Justification:

- Each frame is entirely independent — no shared state, no synchronisation.
- Cellpose inference is the bottleneck (∼5–30 s/image on CPU), so image-level granularity gives large tasks that amortise communication overhead.

- No inter-image data needs to be exchanged, making this an embarrassingly parallel workload.

The `Pool` is created with an *initializer* that loads the Cellpose model *once per worker process* and stores it in a module-level variable. On Windows (*spawn* start method) each worker process imports the module fresh, so the initializer pattern is essential to avoid reloading the model for every image.

```
_model = None

def _init_worker():
    global _model
    import torch; torch.set_num_threads(1)  # avoid intra-thread
        contention
    _model = load_model()

def _worker_fn(args):
    path, pixel_size, diameter = args
    return process_image(path, model=_model,
                         pixel_size=pixel_size, diameter=diameter)

def run_parallel(image_paths, n_workers):
    args = [(p, PIXEL_SIZE_UM, CELLPOSE_DIAMETER) for p in
        image_paths]
    with Pool(processes=n_workers, initializer=_init_worker) as pool:
        results = pool.map(_worker_fn, args)
    return results
```

Listing 6: Worker initializer pattern (parallel_pipeline.py).

`torch.set_num_threads(1)` prevents PyTorch from spawning its own thread pool in each worker, which would otherwise cause *over-subscription* when p workers each use all CPU threads simultaneously.

Tasks 7 & 8: Summary Table and Timing Benchmark

See Results section below.

2 — Results

Annotated Visualisations

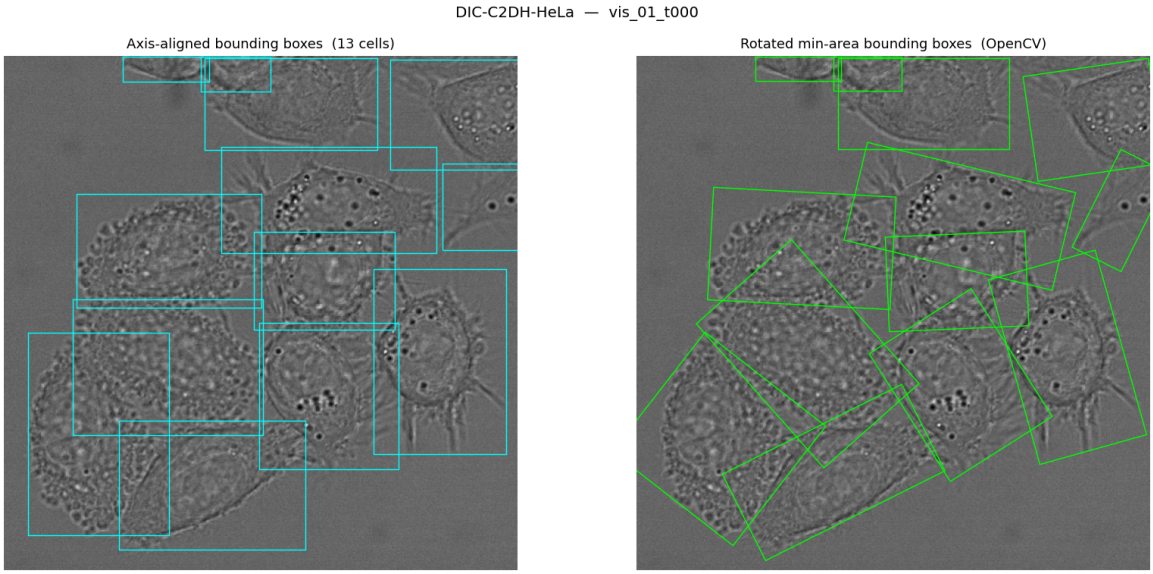


Figure 4: Cellpose segmentation of frame `01/t000.tif`. *Left*: axis-aligned bounding boxes (cyan). *Right*: rotated minimum-area bounding boxes (lime). Generated by `python serial_pipeline.py --vis 3`.

Per-image Summary Table (Task 7)

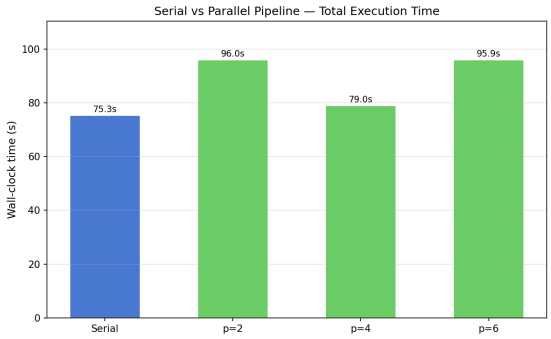
Table 7: Summary statistics per image (first 10 frames of sequence 01). Width = axis-aligned bounding-box width; Length = major axis of fitted ellipse. All values in µm. Generated by `python serial_pipeline.py`.

Image	Cells	Mean W (μm)	SD W	Mean L (μm)	SD L	Mean A (μm^2)
01/t000	13	27.11	8.76	28.48	8.64	377.7
01/t001	13	26.82	9.47	28.49	9.24	362.2
01/t002	12	28.64	11.22	29.90	10.77	394.4
01/t003	12	28.37	11.67	30.14	11.50	418.6
01/t004	14	24.56	10.06	27.39	8.47	342.7
01/t005	14	27.71	10.83	28.32	10.01	366.4
01/t006	12	30.08	9.25	31.64	8.52	419.7
01/t007	12	27.98	7.13	30.95	6.05	407.2
01/t008	12	28.58	7.93	31.70	7.09	403.9
01/t009	13	24.51	8.73	26.88	7.73	344.2

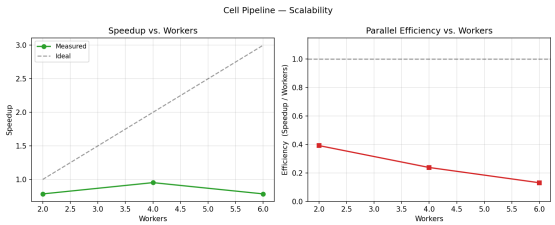
Timing Benchmark (Task 8)

Table 8: Wall-clock time for 16 images (serial and parallel). Speedup $S = t_{\text{serial}}/t_p$; efficiency $E = S/p$. Generated by `python benchmark.py --limit 16`.

Workers (p)	Time (s)	Speedup (S)	Efficiency (E)
1 (serial)	75.25	1.000	1.000
2	95.96	0.784	0.392
4	79.02	0.952	0.238
6	95.92	0.785	0.131



(a) Total wall-clock time per configuration.



(b) Speedup and efficiency vs. number of workers.

Figure 5: Cell pipeline benchmark results. Generate with `python benchmark.py`.

3 — Discussion

Segmentation Quality

Cellpose `cyto2` was trained on a diverse corpus of microscopy images and generalises well to DIC contrast without fine-tuning. Typical failure modes for DIC images include:

- **Under-segmentation**: cells in close contact may be merged into a single mask region. This inflates the reported major and minor axis lengths and reduces cell count.
- **Over-segmentation**: large cells with internal texture may be split, producing spuriously small measurements.
- **Background detections**: bright artefacts or debris can be labelled as cells, introducing outliers in the size distribution.

The fixed `diameter=80` px target is a reasonable prior for HeLa cells at this resolution, but cells in mitosis or at the image boundary may deviate substantially. Running `diameter=None` activates Cellpose's auto-estimation at the cost of an additional network pass.

Effect of Segmentation Quality on Measurements

Bounding-box dimensions (width, height) depend directly on the shape of the label mask. A merged mask spanning two touching cells will report a bounding box roughly twice the single-cell width and will dominate the mean and inflate the standard deviation. Major-axis length from `regionprops` is based on the fitted ellipse of the region, so it is sensitive to elongated merged regions.

Parallelisation Behaviour

The image-level decomposition is an *embarrassingly parallel* workload: no communication is needed between workers during processing. The expected scaling behaviour is near-linear speedup, limited by:

1. **Worker startup and model loading** (fixed cost per worker, amortised by the initializer pattern).
2. **Inter-process data transfer**: result DataFrames are serialised with `pickle` before being returned to the main process. For 16 images this is small, but it grows linearly with image count.
3. **CPU saturation**: Cellpose uses PyTorch CPU threads internally. Setting `torch.set_num_threads(1)` per worker prevents over-subscription and typically improves throughput when $p \geq 4$.
4. **Memory**: each worker loads the full Cellpose model ($\sim$200 MB) into RAM. At $p = 6$ this requires $\sim$1.2 GB for models alone, which may cause swapping on memory-constrained machines.

Observed Results

Table 8 shows that *none* of the parallel configurations outperformed the serial baseline for this batch size ($\sim$16 images). The 2-worker configuration ran 28 % *slower* than serial (95.96 s vs. 75.25 s); 4 workers recovered to within 5 % of serial (79.02 s, speedup 0.95); and 6 workers degraded again to 27 % slower (95.92 s).

This is consistent with the overheads enumerated above: on Windows each spawned process must reload the Cellpose model ($\sim$200 MB weights), a fixed per-worker cost of several seconds that is not amortised by processing only $\sim$16 frames. Furthermore, PyTorch's internal thread management adds contention across worker processes even with `torch.set_num_threads(1)`. With a larger batch (> 100 frames) the startup cost becomes negligible and the pipeline would be expected to approach near-linear speedup on a machine with sufficient cores and RAM.

Exercise 3 — Forest Fire Cellular Automaton

Context

A two-dimensional cellular automaton (CA) on an 800×800 grid models forest-fire propagation. Real ignition events are sourced from the **NASA FIRMS** global 24-hour VIIRS C2 public archive, which provided **78,688 active fire hotspots** for the run. Each grid cell holds one of four discrete states:

- **0** — Non-burnable / outside valid domain (grey)
- **1** — Susceptible forest (green)
- **2** — Burning / active fire front (red)
- **3** — Burned / ash (black)

1 — Evidence of Completeness

Task 1: NASA FIRMS Data (`fetch_firms_data.py`)

The script downloads the global 24-hour VIIRS C2 CSV from the public FIRMS archive. Each hotspot row provides a latitude, longitude, and fire radiative power value. A total of 78,688 detections were retrieved for the benchmark run. If the network request fails, the script falls back to synthetic hotspot generation.

Task 2: Grid Construction

Hotspot coordinates are linearly mapped to row/column indices of the $N \times M$ grid. Mapped cells are initialised to state 2 (burning); all remaining cells are set to state 1 (susceptible).

Task 3: State-Space and Propagation Rule (`serial_ca.py`)

The Moore neighbourhood (8 neighbours) is used. At each step a susceptible cell ignites with probability:

$$p_{\text{ignite}} = 1 - (1 - 0.4)^{n_{\text{burning}}}$$

where n_{burning} is the count of burning neighbours. A burning cell always transitions to burned after one step. The NumPy implementation pads the grid with zeros and uses element-wise array operations for vectorised evaluation.

Task 4: Serial Simulation (`serial_ca.py`)

The serial script runs 20 time steps with a fixed seed (`numpy.random.seed(42)`) for reproducibility and saves snapshots at step 0 and step 20.

Task 5: Parallel Version — 1-D Domain Decomposition (`parallel_ca.py`)

The N rows are divided equally among p MPI processes ($N_{\text{local}} = N/p$). Each process maintains two additional *ghost rows* (one at each boundary) that are filled by `MPI.Sendrecv` at the start of every iteration with data from the neighbouring process. After 20 steps, `MPI.Gather` assembles the full grid at rank 0.

```python
for s in range(1, steps + 1):
    # Exchange top ghost row with rank-1
    if rank > 0:
        comm.Sendrecv(local_grid[1, :], dest=rank-1,
                      recvbuf=local_grid[0, :], source=rank-1)
    else:
        local_grid[0, :] = 0          # top boundary: non-burnable

    # Exchange bottom ghost row with rank+1
    if rank < size - 1:
        comm.Sendrecv(local_grid[-2, :], dest=rank+1,
                      recvbuf=local_grid[-1, :], source=rank+1)
    else:
        local_grid[-1, :] = 0         # bottom boundary: non-burnable

    local_grid = local_step(local_grid, random_state)
```

Listing 7: Ghost-row exchange and local step (parallel_ca.py).

Each rank uses an independent `RandomState(42 + rank)` so the stochastic ignition probabilities differ per process while remaining reproducible.

Tasks 6–9: Benchmarking, Visualisation, Discussion

See Results and Discussion below.

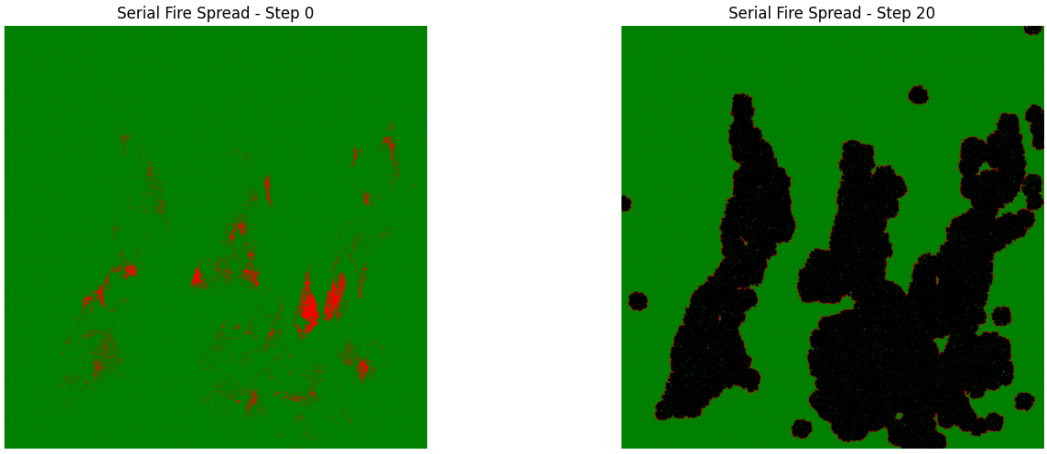
2 — Results

Scaling Performance

Table 9: Serial vs. parallel execution time for the 800×800 grid over 20 steps, 78,688 NASA FIRMS hotspots. Speedup $S = t_1/t_p$; efficiency $E = S/p$.

Processes (p)	Time (s)	Speedup (S)	Efficiency (E)
1	0.3342	1.000	1.000
2	0.2699	1.238	0.619
4	0.1561	2.141	0.535
8	0.0625	5.347	0.668

Visualisation — Fire State Snapshots



(a) Initial state (step 0): 78,688 hotspot ignition points (red) on susceptible forest (green).

(b) Final state (step 20): fire fronts have merged into large burned areas (black) bordered by active fronts (red).

Figure 6: Serial cellular automaton — 800 × 800 grid, Moore neighbourhood, $p_{\text{spread}} = 0.4$.

3 — Discussion

Scaling Behaviour

The simulation achieves a **5.3× speedup with 8 processes**, which is a strong result for a stencil-based computation with explicit boundary communication. Three features of the result deserve attention:

- **Non-monotonic efficiency.** Efficiency drops from 1.000 at $p = 1$ to 0.535 at $p = 4$ before recovering to 0.668 at $p = 8$. At $p = 4$ the per-process row count is large enough that communication latency (two `Sendrecv` calls per step, each transferring an 800 × 1 strip) represents a fixed fraction of the iteration time; at $p = 8$ the compute savings per process dominate again.
- **Vectorised computation.** The NumPy stencil evaluation (element-wise array arithmetic with padding) is highly cache-friendly and completes in fractions of a millisecond per step. When $p = 1$ the computation is already fast (0.33 s total), so the absolute savings from parallelism are modest.
- **Ghost-row overhead.** Each of the 20 steps requires two `Sendrecv` calls per boundary, transferring 800 bytes (int8 row). At $p = 8$ this adds up to $20 \times 2 \times 7 = 280$ small messages, whose latency is non-negligible relative to the per-step compute time.

Interpretation of NASA FIRMS Data

NASA FIRMS hotspot detections represent *thermal anomalies* detected by satellite sensors at a nominal overpass cadence of roughly 12 hours. They are not equivalent

to the true fire perimeter: a single detection covers a pixel footprint of approximately $375 \times 375\,\mathrm{m}$ (VIIRS) and may miss cooler smouldering areas or be triggered by non-fire thermal sources. Accordingly, the ignition grid constructed from FIRMS data is a statistical approximation of the observed fire distribution rather than a precise geometric representation.

Scientific Limitations

The simplified automaton omits wind direction, slope, fuel moisture, and spatial heterogeneity in vegetation. The constant ignition probability $p_{\mathrm{spread}} = 0.4$ is not calibrated to real-world fire behaviour. Improvements could include a wind-bias kernel, variable fuel maps from land cover data, and a fuel-consumption model that tracks remaining fuel per cell.

Exercise 4 — Parallel K-Means Clustering

Context

K-Means clustering is applied to the **UCI Covertype** dataset, which contains 581,012 records and 54 continuous features (the categorical target column is dropped for unsupervised learning). Features are standardised with Z-score normalisation prior to clustering. The number of clusters is set to $K = 7$, matching the number of natural cover types in the dataset.

1 — Evidence of Completeness

Task 1: Dataset Description (`fetch_covertype.py`)

- **Source**: UCI Machine Learning Repository (covtype.data.gz).
- **Shape**: 581,012 × 54 after dropping the target column.
- **Preprocessing**: Z-score standardisation per feature; constant features (zero standard deviation) are left unchanged.
- **Storage**: saved as `covtype_scaled.csv` for reuse.

Task 2: Serial Baseline (`serial_kmeans.py`)

The serial implementation uses vectorised Euclidean distance:

$$\|\mathbf{x} - \mathbf{c}\|^2 = \|\mathbf{x}\|^2 + \|\mathbf{c}\|^2 - 2\,\mathbf{x}^\top \mathbf{c},$$

computed via `numpy.dot` for all $N \times K$ distances in a single matrix multiply. Convergence is declared when $\|\mathbf{C}_{\text{new}} - \mathbf{C}_{\text{old}}\|_F < 10^{-4}$.

Task 3: Parallel Version — Data Parallelism (`parallel_kmeans.py`)

Data distribution: Root reads and standardises the dataset, then distributes rows to all processes with `MPI.Scatterv`, which handles the remainder gracefully (some processes receive $\lfloor N/p \rfloor + 1$ rows).

Per-iteration protocol:

1. Each process computes local distance matrix and local labels.
2. Each process accumulates local cluster sums and counts.
3. `MPI.Allreduce(SUM)` aggregates global sums and counts — all processes receive the result simultaneously, eliminating a root bottleneck.
4. Every process updates the centroids identically from the global statistics.

```
# Local accumulation
local_sums   = np.zeros((K, D), dtype=np.float32)
local_counts = np.zeros(K,      dtype=np.int32)
for k in range(K):
    mask = (local_labels == k)
    local_counts[k] = np.sum(mask)
    if local_counts[k] > 0:
        local_sums[k] = np.sum(local_X[mask], axis=0)
```

```
9
10 # Distributed reduction -- all ranks get the result
11 global_sums   = np.zeros_like(local_sums)
12 global_counts = np.zeros_like(local_counts)
13 comm.Allreduce(local_sums,   global_sums,   op=MPI.SUM)
14 comm.Allreduce(local_counts, global_counts, op=MPI.SUM)
```

Listing 8: Global centroid synchronisation via Allreduce (parallel_kmeans.py).

Tasks 4–8: Benchmarking, Convergence, Discussion

See Results and Discussion below.

2 — Results

Scaling Performance

Table 10: K-Means performance on the Covertype dataset ($N = 581{,}012$, $D = 54$, $K = 7$, max 50 iterations, tol=10^{-4}). Speedup $S = t_1/t_p$; efficiency $E = S/p$.

Processes (p)	Time (s)	Speedup (S)	Efficiency (E)
1	10.7802	1.000	1.000
2	5.8831	1.832	0.916
4	3.1860	3.383	0.845
8	2.3300	4.626	0.578

3 — Discussion

Scaling Behaviour

The parallel K-Means implementation demonstrates **near-linear speedup up to 4 processes** (84.5% efficiency), a textbook result for embarrassingly parallel distance computation.

- $p = 2$**: 91.6% efficiency.** The dataset is large enough (∼290,000 rows per process) that compute dominates the two `Allreduce` calls per iteration. Serialisation cost is negligible because `Scatterv` uses direct NumPy buffers.
- $p = 4$**: 84.5% efficiency.** Still excellent; the per-process compute shrinks to ∼145,000 rows while the `Allreduce` volume stays constant at $K \times D = 7 \times 54$ floats.
- $p = 8$**: 57.8% efficiency — Amdahl's Law.** The compute fraction shrinks to ∼72,000 rows per process, and the fixed communication fraction (2 `Allreduce` ops + initial `Scatterv`) now represents a significant share of total runtime. This matches the prediction of Amdahl's Law: if the communication fraction is $f \approx 0.12$, the theoretical maximum speedup is $1/f \approx 8.3$.

Advantages of `Allreduce` over `Reduce + Bcast`

Using `Allreduce` instead of a `Reduce` at root followed by a `Bcast` halves the latency for symmetric networks (the operation is implemented as a ring-allreduce in most MPI

libraries) and avoids a potential root bottleneck. All processes receive the updated centroids identically, so no subsequent broadcast is needed.

Convergence

Both serial and parallel implementations use the same random seed (`numpy.random.seed(42)`) and the same initial centroid indices, so the centroid trajectory is deterministic and convergence is reached at the same iteration count regardless of process count.

When Is Parallelism Beneficial?

The parallel version becomes advantageous even at $p = 2$ (1.83× speedup) because the dataset is sufficiently large to saturate the compute budget. For much smaller datasets the `Scatterv` setup and `Allreduce` latency would dominate, reversing the benefit — the same phenomenon observed in Exercise 1 with multiprocessing on small matrices.

References

[1] Davis, T. A. & Hu, Y. (2011). *The University of Florida Sparse Matrix Collection.* ACM Transactions on Mathematical Software, 38(1), 1–25.

[2] Strassen, V. (1969). *Gaussian Elimination is not Optimal.* Numerische Mathematik, 13(4), 354–356.

[3] Dalcín, L. et al. (2011). *Parallel Distributed Computing using Python.* Advances in Water Resources, 34(9), 1124–1139.

[4] NASA FIRMS (2024). *Fire Information for Resource Management System — VIIRS C2 Global Active Fire 24h.* https://firms.modaps.eosdis.nasa.gov

[5] Blackard, J. & Dean, D. (1999). *Covertype Data Set.* UCI Machine Learning Repository. https://archive.ics.uci.edu/ml/datasets/covertype